

VPS-Net Paper Review

1. Introduction

This paper addresses the problem of vacant parking slot detection for Park Assist Systems (PAS) using Around View Monitor (AVM) images. Existing vision-based approaches are often affected by:

- Lighting variations
- Shadows
- Reflections
- Limited camera field of view
- Complex outdoor environments

Traditional marking point-based methods require complicated geometric matching, while semantic segmentation methods require computationally expensive post-processing. To overcome these limitations, the authors propose VPS-Net, a two-stage deep learning framework that combines parking slot detection and occupancy classification for accurate and real-time parking slot recognition.

2. Related Work

The paper reviews previous studies in two categories.

2.1 Parking Slot Detection

Line-based Methods

- Detects parking slot lines using image processing.
- Sensitive to shadows, and noise.

Marking Point-based Methods

Advantages:

- Better localization accuracy.

Limitations:

- Require complex matching algorithms.
- Cannot determine parking slot occupancy.

Segmentation-based Methods

Advantages:

- Accurate segmentation of parking markings.

Limitations:

- Require post-processing to generate parking slots.
- Increased computational cost and inference time.

2.2 Parking Slot Occupancy Classification

Earlier occupancy classification methods depended on manually designed image features.

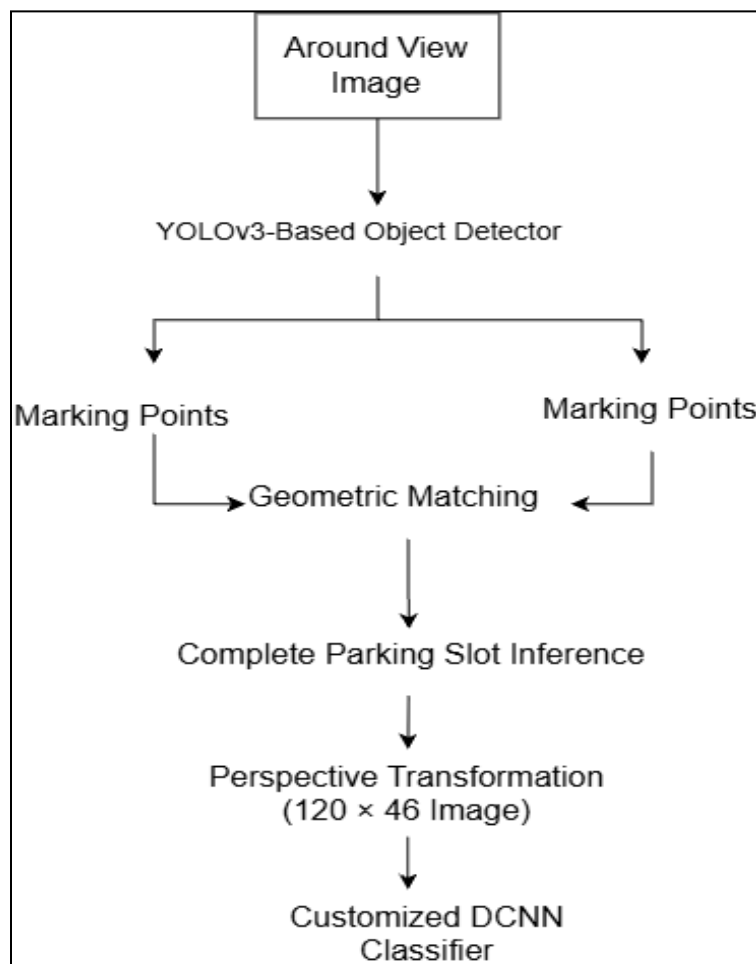
Recent approaches adopted deep learning models including:

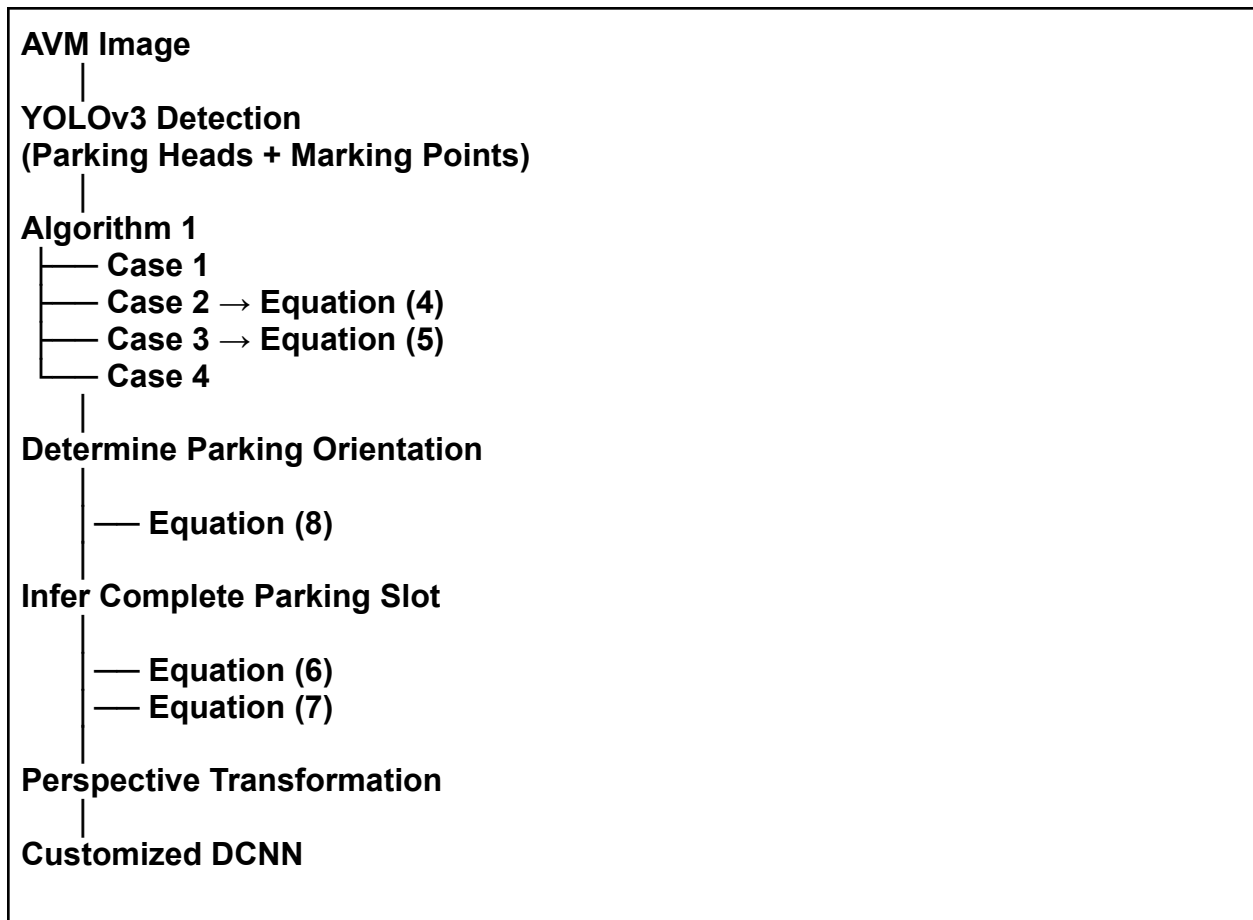
- AlexNet
- VGG
- ResNet
- MobileNet

Although these CNN models provide high accuracy, they contain many parameters and are relatively large for a simple binary occupancy classification task.

3. Methodology

The proposed VPS-Net is divided into two sequential modules.





Module 1: Parking Slot Detection

The first module detects the location and type of parking slots.

The process consists of:

Step 1: YOLOv3 Detection

A YOLOv3 detector simultaneously detects:

- Parking slot heads
- T-shaped marking points
- L-shaped marking points

Unlike previous methods that only detect marking points, VPS-Net predicts both parking heads and marking points, reducing the complexity of subsequent matching.

Step 2: Marking Point Matching

After detection, the system applies geometric rules to determine paired marking points.

The algorithm handles different situations:

- Both marking points detected
- Only one marking point detected
- No marking points detected
- Multiple marking points detected

When some marking points are missing, VPS-Net estimates their positions using the detected parking slot head and image intensity information.

Step 3: Complete Parking Slot Inference

Using the paired marking points, VPS-Net calculates:

- Parking slot type
- Parking orientation
- Two invisible corner points

This allows reconstruction of the complete parking slot even when only part of it is visible in the AVM image.

Module 2: Occupancy Classification

After the parking slot is detected:

1. The parking slot is cropped.
2. Perspective transformation normalizes it to 120×46 pixels.
3. The normalized image is input to a customized DCNN.

Compared with standard CNN architectures, the customized DCNN:

- Uses fewer convolution layers.
- Uses fewer filters.
- Has fewer parameters.
- Provides faster inference while maintaining high classification accuracy.

The network finally classifies the parking slot as:

- Vacant
- Occupied

4. Experimental Setup

Datasets

Two public datasets were used:

ps2.0

- 12,165 AVM images

- 9,827 training images
- 2,338 testing images

PSV

- Used to evaluate model generalization.

Since neither dataset originally contained occupancy annotations, the authors manually labeled parking slot occupancy for training and testing.

Implementation

- Framework: PyTorch
- CPU: Intel Core i9-7900X
- GPU: Dual NVIDIA Titan Xp
- Memory: 32 GB RAM

5. Results

5.1 Parking Slot Detection

Method	Precision	Recall
PSD_L	98.55%	84.89%
DeepPS	99.77%	98.62%
VPS-Net	99.58%	99.26%

5.2 Occupancy Classification

Model	Accuracy	Runtime	Model Size
HOG + SVM	92.54%	2.13 ms	0.04 MB
AlexNet	99.67%	1.75 ms	228.1 MB
Customized DCNN	99.48%	0.81 ms	9.4 MB

The customized DCNN achieved accuracy comparable to AlexNet while significantly reducing runtime and model size.